Crossword Application — Restructuring Plan

> **Version:** 1.0 · **Date:** 2026-03-19 · **Status:** Proposed

---

Table of Contents

1. Motivation
2. Architecture Overview
3. Directory Structure
4. Implementation Strategy
5. Critical Files
6. REST API Contract
7. Data Models & DTOs
8. Key Design Decisions
9. Testing Plan
10. Migration Path
11. Risks & Mitigations
12. Open Questions

---

1. Motivation

Problems with the Current Architecture

The current application is a **monolithic Flask app** with server-side sessions and SQLAlchemy ORM. While functional, it has a number of structural problems that compound as the application grows.

Problem	Impact
Flask blueprints contain **business logic** mixed with HTTP plumbing	Hard to test; changes to one layer force changes in the other
Server-side sessions store entire domain objects (Grid, Puzzle JSON)	Fragile; session corruption loses all in-progress work
SQLAlchemy abstracts over SQLite — the only DB ever used	Adds dependency weight and `db.session` boilerplate for no benefit
All rendering is **server-side Jinja2**	Every user action requires a full page reload; no partial updates
Domain models (`Grid`, `Puzzle`, `Word`) are **already pure Python**	They're ready for hexagonal architecture but are buried under UI glue
`userid=1` is **hardcoded everywhere**	Any future multi-user support requires touching every query
Export logic (`puzzle_to_xml.py`, etc.) lives inside `crossword/ui/`	Domain-level concerns mixed into the web delivery layer

Goals

1. **Remove Flask** — replace with Python's built-in `http.server`; no web framework dependency.
2. **Remove SQLAlchemy** — replace with direct `sqlite3` calls inside dedicated adapter classes.
3. **Hexagonal architecture (Ports & Adapters)** — the domain must not know anything about HTTP, SQL, or files.
4. **Plain-JavaScript SPA frontend** — full-page reloads replaced by `fetch()` calls; jQuery allowed for DOM helpers.
5. **Preserve all existing domain tests** — `crossword/tests/` must pass without modification.

---

2. Architecture Overview

Conceptual Layers

```
┌──────────────────────────────────────────────────────────────┐
│                    Frontend (Browser)                        │
│  - Plain JavaScript (optional jQuery)                        │
│  - REST API client via fetch()                               │
│  - Client-side state (current grid/puzzle in memory)         │
└──────────────────────┬───────────────────────────────────────┘
                       │ HTTP / JSON
┌──────────────────────▼───────────────────────────────────────┐
│              HTTP Adapter  (Driving Adapter)                 │
│  - stdlib http.server + custom RequestHandler                │
│  - URL routing, request parsing, response serialisation      │
│  - Session token in header (simple, stateless server)        │
└──────────────────────┬───────────────────────────────────────┘
                       │ calls Use Cases
┌──────────────────────▼───────────────────────────────────────┐
│              Application / Use Cases Layer                   │
│  - One class per feature area (GridUseCases, etc.)           │
│  - Orchestrates domain objects and ports                     │
│  - Enforces business workflows and transactions              │
└──────────────────────┬──────────────────────────────────────┬┘
   │ uses domain model                                        │ uses ports
┌──▼──────────────────────┐    ┌──────────────────────▼──────┐
│   Domain Model          │    │  Port Interfaces (Abstract) │
│  ─────────────────      │    │  ──────────────────────     │
│  Grid, Puzzle, Word     │    │  PersistencePort            │
│  NumberedCell           │    │  WordListPort               │
│  Validation rules       │    │  ExportPort                 │
│                         │    │  FileStoragePort            │
│  (PURE — zero deps)     │    │  (ABC — zero deps)          │
└─────────────────────────┘    └──────────────────────┬──────┘
                                                      │ implemented by
                               ┌──────────────────────▼──────┐
                               │  Adapters  (Driven Adapters) │
                               │  ──────────────────────     │
                               │  SQLiteAdapter              │
                               │  JSONFileAdapter            │
                               │  DictionaryAdapter          │
                               │  AcrossLiteAdapter          │
                               │  CrosswordCompilerAdapter   │
                               │  NYTimesAdapter             │
                               └─────────────────────────────┘
```

Current vs. New — At a Glance

Concern	Current	New
HTTP framework	Flask 3.x	`http.server` (stdlib)
ORM / DB access	Flask-SQLAlchemy	`sqlite3` (stdlib)
Session storage	Filesystem files (Flask-Session)	Stateless server; client holds IDs
Templating	Jinja2 server-side	None — static HTML + JS
Frontend	Jinja2 + redirect-based JS	Single-page app with `fetch()`
Business logic location	Flask blueprints	Use-case classes
Export logic location	`crossword/ui/`	`crossword/adapters/export/`
External dependencies	Flask, Flask-Session, Flask-SQLAlchemy	**Zero** (stdlib only)

---

3. Directory Structure

```
crossword/
│
├── domain/                              # Pure domain logic (UNCHANGED)
│   ├── __init__.py
│   ├── grid.py
│   ├── puzzle.py
│   ├── word.py
│   ├── numbered_cell.py
│   └── letter_list.py
│
├── ports/                               # Abstract port interfaces (NEW)
│   ├── __init__.py
│   ├── persistence.py                   # PersistencePort (grids, puzzles, users)
│   ├── word_list.py                     # WordListPort
│   ├── export.py                        # ExportPort
│   └── file_storage.py                  # FileStoragePort
│
├── adapters/                            # Concrete implementations (NEW)
│   ├── persistence/
│   │   ├── sqlite_adapter.py            # Direct sqlite3 — no ORM
│   │   └── json_adapter.py             # JSON-file alternative
│   ├── word_list/
│   │   └── dictionary_adapter.py       # Loads words.txt / DB table
│   └── export/
│       ├── acrosslite_adapter.py       # Moved from ui/
│       ├── cw_compiler_adapter.py      # Moved from ui/
│       ├── nytimes_adapter.py          # Moved from ui/
│       └── svg_adapter.py             # Moved from root
│
├── use_cases/                           # Application orchestration (NEW)
│   ├── grid_use_cases.py
│   ├── puzzle_use_cases.py
│   ├── word_use_cases.py
│   └── export_use_cases.py
│
├── http_server/                         # HTTP delivery layer (NEW)
│   ├── server.py                        # BaseHTTPRequestHandler subclass + router
│   ├── handlers/
│   │   ├── grid_handlers.py
│   │   ├── puzzle_handlers.py
│   │   ├── word_handlers.py
│   │   ├── export_handlers.py
│   │   └── static_handlers.py          # Serve index.html, JS, CSS
│   └── middleware/
│       ├── request_parser.py
│       ├── response_formatter.py
│       └── error_handler.py
│
├── frontend/                            # Static SPA assets (REFACTORED)
│   ├── index.html                       # Single entry point
│   ├── js/
```

```
│   │   ├── api.js                      # fetch() wrapper for REST API
│   │   ├── app.js                      # Bootstrap + routing
│   │   ├── state.js                    # Client-side state (current grid/puzzle)
│   │   ├── grid-editor.js
│   │   ├── puzzle-editor.js
│   │   ├── word-editor.js
│   │   └── dialogs.js                 # Reusable modal dialogs
│   ├── css/
│   │   ├── style.css
│   │   └── editors.css
│   └── lib/
│       └── jquery.min.js              # Optional
│
├── config/                             # Configuration (NEW)
│   ├── settings.py                     # Reads ~/.crossword.ini
│   └── wiring.py                       # Dependency injection — wires adapters into use cases
│
├── tests/                              # Existing tests (UNCHANGED)
│   └── test_*.py
│
└── main.py                             # Entry point (REFACTORED)
```

---

4. Implementation Strategy

Phase 1 — Ports & Adapters Core (Days 1–2)

1. Define port interfaces (`crossword/ports/`)

Each port is an abstract base class (`abc.ABC`) whose methods declare what the application needs from the outside world — no implementation, no imports from infrastructure.

```
# ports/persistence.py
from abc import ABC, abstractmethod
from domain.grid import Grid

class PersistencePort(ABC):
    @abstractmethod
    def save_grid(self, userid: int, name: str, grid: Grid) -> None: ...

    @abstractmethod
    def load_grid(self, userid: int, name: str) -> Grid: ...

    @abstractmethod
    def list_grids(self, userid: int) -> list[str]: ...

    @abstractmethod
    def delete_grid(self, userid: int, name: str) -> None: ...
```

2. Implement adapters (`crossword/adapters/`)

- `SQLiteAdapter` — `sqlite3` with the same schema as today (`grids` , `puzzles` , `words` , `users` tables), but accessed directly without SQLAlchemy.
- `DictionaryAdapter` — loads the word list into memory once; same behaviour as current.
- Export adapters — move `puzzle_to_xml.py` , `puzzle_publish_acrosslite.py` , `puzzle_publish_nytimes.py` from `crossword/ui/` into `crossword/adapters/export/` .

3. Create use-case classes (`crossword/use_cases/`)

Each class receives its ports via constructor injection and exposes one method per business operation.

```
# use_cases/grid_use_cases.py
class GridUseCases:
    def __init__(self, persistence: PersistencePort):
        self._db = persistence

    def create_grid(self, userid: int, size: int) -> Grid:
        grid = Grid(size)
        return grid

    def save_grid(self, userid: int, name: str, grid: Grid) -> None:
        self._db.save_grid(userid, name, grid)

    def toggle_black_cell(self, grid: Grid, r: int, c: int) -> Grid:
        grid.toggle_black(r, c)
        return grid
```

4. Wire dependencies (`crossword/config/wiring.py`)

The entry point (`main.py`) calls `wiring.py` which instantiates adapters and injects them into use cases. No use-case class ever imports a concrete adapter directly.

Phase 2 — HTTP Server (Days 3–4)

1. Custom request handler (`http_server/server.py`)

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class CrosswordHandler(BaseHTTPRequestHandler):
    # Router: map (method, path pattern) -> handler function
    ...
```

Simple regex-based router maps `(method, path)` pairs to handler functions. No third-party routing library needed.

2. Handler functions (`http_server/handlers/`)

Each handler: 1. Parses the request (URL params or JSON body) 2. Calls the appropriate use-case method 3. Serialises the result to JSON and writes the response

3. Static file serving

`static_handlers.py` serves `frontend/` assets (HTML, JS, CSS) with correct `Content-Type` headers. A single `GET /` route returns `index.html`.

4. Session simplification

Flask-Session is eliminated. The server is stateless. The browser tracks: - The current grid ID / name - The current puzzle ID / name - The current UI mode

A minimal session token (UUID in a cookie) is used only to identify the user row in the DB (still `userid=1` for now).

Phase 3 — Frontend SPA (Days 5–6)

1. **`index.html`** — one file with placeholder `<div>` elements for each view: - `#grid-editor`, `#puzzle-editor`, `#word-editor` - `#toolbar`, `#menu`, `#dialog-overlay`

2. **`api.js`** — thin `fetch()` wrapper:

```
const API = {
  getGrids: () => fetch('/api/grids').then(r => r.json()),
  saveGrid: (id, data) => fetch(`/api/grids/${id}`, {
      method: 'PUT',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify(data)
  }).then(r => r.json()),
  // ...
};
```

3. **`state.js`** — a simple observable store:

```
const State = (() => {
  let _state = { mode: 'MAIN', grid: null, puzzle: null };
  const _listeners = [];
  return {
    get: () => ({..._state}),
    set: (patch) => { _state = {..._state, ...patch}; _listeners.forEach(fn => fn(_state)); },
    subscribe: (fn) => _listeners.push(fn),
  };
})();
```

4. Editor modules — `grid-editor.js`, `puzzle-editor.js`, `word-editor.js` each: - Subscribe to `State` - Render the SVG / UI elements into their placeholder `<div>` - Attach event listeners (clicks on SVG, button clicks) - Call `API.*` to persist changes back to the server

jQuery may be used for convenience in DOM manipulation; it is not mandatory.

Phase 4 — Testing & Cutover (Day 7)

1. **Run existing tests** — confirm `crossword/tests/` passes unchanged (domain is untouched).
2. **Add adapter tests** — test `SQLiteAdapter` CRUD operations against an in-memory SQLite DB.
3. **Add use-case tests** — inject mock ports; verify business rules.
4. **Add HTTP handler tests** — send raw HTTP requests; verify JSON responses.
5. **Manual end-to-end walkthrough** — full workflow: create grid → puzzle → edit → export.

---

5. Critical Files

New Files to Create

File	Purpose	Est. Lines
`crossword/ports/persistence.py`	`PersistencePort` ABC	~80
`crossword/ports/word_list.py`	`WordListPort` ABC	~25
`crossword/ports/export.py`	`ExportPort` ABC	~30
`crossword/adapters/persistence/sqlite_adapter.py`	`sqlite3` implementation	~220
`crossword/adapters/word_list/dictionary_adapter.py`	Word list loader	~80
`crossword/use_cases/grid_use_cases.py`	Grid operations	~130
`crossword/use_cases/puzzle_use_cases.py`	Puzzle operations	~180
`crossword/use_cases/word_use_cases.py`	Word/clue operations	~90
`crossword/use_cases/export_use_cases.py`	Export orchestration	~60
`crossword/http_server/server.py`	HTTP server + router	~150
`crossword/http_server/handlers/grid_handlers.py`	Grid routes	~140
`crossword/http_server/handlers/puzzle_handlers.py`	Puzzle routes	~140
`crossword/http_server/handlers/word_handlers.py`	Word routes	~70
`crossword/http_server/handlers/export_handlers.py`	Export routes	~60
`crossword/config/wiring.py`	Dependency wiring	~50
`crossword/frontend/index.html`	SPA shell	~60
`crossword/frontend/js/api.js`	REST client	~100
`crossword/frontend/js/state.js`	Client state	~40
`crossword/frontend/js/grid-editor.js`	Grid UI	~200
`crossword/frontend/js/puzzle-editor.js`	Puzzle UI	~200

Files to Refactor

File	Changes Required
`crossword/__init__.py`	Remove SQLAlchemy config; keep domain exports
`crossword/ui/__init__.py`	Delete Flask app factory and DB models
`crossword/ui/main.py`	Replace with `crossword/main.py` using new HTTP server
`crossword/ui/puzzle_to_xml.py`	Move to `adapters/export/cw_compiler_adapter.py`
`crossword/ui/puzzle_publish_acrosslite.py`	Move to `adapters/export/acrosslite_adapter.py`
`crossword/ui/puzzle_publish_nytimes.py`	Move to `adapters/export/nytimes_adapter.py`
`requirements.txt`	Remove Flask, Flask-Session, Flask-SQLAlchemy

Files to Delete

- All of `crossword/ui/templates/` (Jinja2 templates)
- `crossword/ui/uimain.py`, `uigrid.py`, `uipuzzle.py`, `uiword.py`, `uiwordlists.py`, `uipublish.py`
- `crossword/ui/uistate.py` (UIState enum replaced by client-side state)
- `crossword/flask_session/` directory

---

6. REST API Contract

Grids

```
GET    /api/grids                      List all grids (name, id, modified)
POST   /api/grids                      Create grid  { "size": 15 }
GET    /api/grids/{name}               Get grid detail + SVG
PUT    /api/grids/{name}               Save grid state
DELETE /api/grids/{name}               Delete grid
POST   /api/grids/{name}/toggle        Toggle black cell  { "r": 3, "c": 5 }
POST   /api/grids/{name}/rotate        Rotate grid 90°
GET    /api/grids/{name}/undo          Undo last black-cell toggle
GET    /api/grids/{name}/redo          Redo
GET    /api/grids/{name}/preview       SVG string
GET    /api/grids/{name}/stats         Statistics JSON
```

Puzzles

```
GET    /api/puzzles                     List all puzzles
POST   /api/puzzles                     Create from grid  { "gridname": "...", "name": "..." }
GET    /api/puzzles/{name}              Get puzzle + SVG + across/down word lists
PUT    /api/puzzles/{name}              Save puzzle (title, etc.)
DELETE /api/puzzles/{name}              Delete puzzle
POST   /api/puzzles/{name}/click        Click cell  { "r": 3, "c": 5 }
GET    /api/puzzles/{name}/undo         Undo
GET    /api/puzzles/{name}/redo         Redo
GET    /api/puzzles/{name}/preview      SVG string
GET    /api/puzzles/{name}/stats        Statistics
```

Words

```
GET    /api/puzzles/{name}/words/{seq}/{dir}             Get word detail
PUT    /api/puzzles/{name}/words/{seq}/{dir}             Save word text + clue
GET    /api/puzzles/{name}/words/{seq}/{dir}/constraints Get per-cell letter constraints
POST   /api/puzzles/{name}/words/{seq}/{dir}/reset       Clear non-constrained letters
GET    /api/wordlist?pattern=..A..                       Get matching word suggestions
```

Exports

```
GET    /api/puzzles/{name}/export/acrosslite            Download .txt
GET    /api/puzzles/{name}/export/compiler-xml          Download .xml
GET    /api/puzzles/{name}/export/nytimes               Download .zip
GET    /api/puzzles/{name}/export/json                  Download raw JSON
```

Static / UI

```
GET    /                                Serve frontend/index.html
GET    /js/{file}                       Serve JS
GET    /css/{file}                      Serve CSS
GET    /lib/{file}                      Serve vendor JS
```

---

7. Data Models & DTOs

The server returns JSON. The domain objects (`Grid`, `Puzzle`) are never sent directly; they are mapped to lightweight DTOs in the handler layer.

Grid Response DTO

```
{
  "name": "my-grid",
  "size": 15,
  "black_cells": [[1,1],[1,15]],
  "numbered_cells": [
    { "seq": 1, "r": 1, "c": 1, "across_len": 5, "down_len": 7 }
  ],
  "svg": "<svg>...</svg>",
  "is_dirty": false,
  "modified": "2026-03-19T10:30:00Z"
}
```

Puzzle Response DTO

```
{
  "name": "my-puzzle",
  "title": "Sunday Puzzle",
  "grid_name": "my-grid",
  "size": 15,
  "cells": { "1,1": "H", "1,2": "E" },
  "across_words": [
    { "seq": 1, "length": 5, "text": "HELLO", "clue": "A greeting", "complete": true }
  ],
  "down_words": [ ... ],
  "svg": "<svg>...</svg>",
  "modified": "2026-03-19T10:30:00Z"
}
```

---

8. Key Design Decisions

1. Database: direct `sqlite3` vs. JSON files

Decision: `SQLiteAdapter` using `sqlite3` (stdlib).

- Same schema as today — migration is a no-op (or simple script).
- SQL queries are more capable than scanning JSON files.
- No extra dependencies; `sqlite3` is always in the stdlib.
- `JSONFileAdapter` will also be implemented as an alternative (useful for testing and simple deployments).

2. HTTP library: `http.server` vs. Bottle

Decision: Python's built-in `http.server`.

- Zero new dependencies.

- Sufficient for a single-user local tool.
- A regex-based router in ~50 lines covers all needed routes.
- Can be swapped for Bottle/Starlette if needed without changing use cases or domain.

3. Frontend state: client-side vs. server sessions

Decision: Stateless server; browser holds current grid/puzzle name in JS variables.

- Eliminates Flask-Session and its filesystem session files.
- Server only stores persistence (DB) — no per-request session state.
- Browser state is ephemeral (page refresh resets to "choose a grid/puzzle"). That is acceptable for a local tool.
- `localStorage` can be added later if page-refresh persistence is desired.

4. Dependency injection: constructor injection

Decision: Use cases receive ports in their constructor; `wiring.py` assembles everything.

```
# config/wiring.py
def make_app(config):
    adapter    = SQLiteAdapter(config.db_path)
    word_list  = DictionaryAdapter(config.word_list_path)
    grid_uc    = GridUseCases(adapter)
    puzzle_uc  = PuzzleUseCases(adapter)
    word_uc    = WordUseCases(adapter, word_list)
    export_uc  = ExportUseCases(adapter)
    return CrosswordHandler(grid_uc, puzzle_uc, word_uc, export_uc)
```

No global state. Easy to test by substituting mock ports.

5. SVG generation: server-side (unchanged)

SVG rendering stays in `to_svg.py`. The server returns SVG strings in API responses; the browser inserts them directly into the DOM. This avoids reimplementing the grid-drawing logic in JavaScript.

---

9. Testing Plan

Existing Tests (Domain — Unchanged)

```
pytest crossword/tests/
```

All 22 existing test modules test pure domain classes with no Flask/SQLAlchemy dependency. They must pass **without modification** after the restructuring.

New Tests

Suite	Location	Coverage
Adapter tests	`crossword/tests/adapters/`	SQLite CRUD, JSON round-trip
Use-case tests	`crossword/tests/use_cases/`	Business rules with mock ports
HTTP handler tests	`crossword/tests/http/`	Route dispatch, JSON responses
Integration tests	`crossword/tests/integration/`	Full flow: create → edit → export

Manual Smoke Test Checklist

- [ ] Start server, open `http://localhost:5000`
- [ ] Create a new 15×15 grid
- [ ] Toggle black cells (symmetric placement enforced)
- [ ] Rotate grid 90°
- [ ] Undo / redo black-cell operations
- [ ] Create a puzzle from the grid
- [ ] Click a cell; enter word text and clue
- [ ] Word suggestions appear matching the letter pattern
- [ ] Save puzzle with a name; close and reopen it
- [ ] Export to AcrossLite (`.txt`) and verify file contents
- [ ] Export to Crossword Compiler XML; verify file
- [ ] Import a puzzle from AcrossLite
- [ ] Validate puzzle (NY Times rules); confirm error messages
- [ ] Check grid statistics page

---

10. Migration Path

Strategy: run both systems in parallel, migrate one feature at a time

```
Week 1  ─┬─ Build ports, adapters, use cases alongside existing Flask app
         │  (Flask app still functional throughout)
         │
Week 2  ─┬─ Build HTTP server and handlers
         │  Test new API endpoints alongside Flask routes
         │
Week 3  ─┬─ Build SPA frontend; connect to new API
         │  Feature-by-feature: grids → puzzles → words → export
         │
Week 4  ─┬─ Data migration (verify round-trip: old DB → new adapter → old DB)
         └─ Delete Flask code; remove old templates; update requirements.txt
```

Data Migration Script

The existing `samples.db` SQLite file uses a schema compatible with the new adapter. The migration script verifies:

1. All `grids` rows load correctly via `Grid.from_json(row.jsonstr)`.
2. All `puzzles` rows load correctly via `Puzzle.from_json(row.jsonstr)`.
3. SHA-256 of each loaded object matches the stored SHA (data integrity).

---

11. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
`http.server` thread-safety issues under concurrent requests	Low (single-user tool)	Medium	Use `ThreadingHTTPServer`; document single-user limitation
SVG click coordinates differ between server-rendered SVG and browser-inserted SVG	Medium	High	Keep existing coordinate calculation; test click handlers early in Phase 3
Client-side state lost on page refresh	Low (accepted trade-off)	Low	Use `sessionStorage` to persist current grid/puzzle name across refresh
Data migration corrupts a puzzle	Low	High	Run migration on a copy; validate SHA before deleting original row
Port interface too narrow — missing methods discovered mid-implementation	Medium	Medium	Iterate port interfaces early; add methods before freezing adapters
jQuery bloat pulling in unwanted patterns	Low	Low	Keep jQuery strictly as a DOM helper; all app logic stays in plain JS

---

12. Open Questions

1. **Authentication** — `userid=1` is still hardcoded in this plan. A real multi-user version would need session tokens and a proper `UserPort`. Defer until after basic restructuring is stable.
2. **Offline / PWA** — Could add `localStorage` or IndexedDB caching for offline editing. Not in scope here.
3. **WebSocket collaboration** — Real-time co-editing is possible once the server is stateless. Out of scope.
4. **Backend framework upgrade** — If `http.server` limitations become a problem (e.g., async needed, proper middleware), the use-case layer is already framework-agnostic; dropping in Bottle or Starlette is a handler-layer change only.
5. **PostgreSQL** — SQLite is fine for a local tool. The `PersistencePort` interface means swapping to PostgreSQL later is an adapter-level change only.